DURHAM COLLEGE

COSC 1100

Class Exercise

COSC 1100
Class Exercise

Arrays: Learning to Use Arrays Effectively

Part 1 of 2

Arrays:
Learning to Use Arrays Effectively

Goal of Part 1:

Experience working with arrays in a program.

Requirements

- Did you come prepared?
  - Did you watch the Lecture videos on arrays?
  - Hopefully you are comfortable with loops!

- Collaboration.
  - You will be encouraged to work together with 1-2 colleagues during this exercise. (i.e. groups of 2-3)
  - The group will share their planning ideas at the end of this class, which will contribute to your grade.

Special Array function

- This special function is the list(), which creates an empty array
- An empty array is still an array, but its size is 0
- This is how we initialize an array

```
# An Empty List.
os_list = list()

os_list.append(‘Windows’)
os_list.append(‘Linux’)
os_list.append(‘Mac’)
Os_list.append(‘Unix’)
```

Zero-Based Index

- Array indexes in many (but not all) programming languages, including Python, start with 0
- The first element in a Python list is called “index 0”
- If the length of an array is 4, the indexes are 0, 1, 2, 3
- Using index 4 on an array like that produces an error

```
# List with length of 4, index 0 through 3
integer_list = [1, 2, 3, -1]
```

DURHAM COLLEGE

Accessing a list's Length

- The *len*() function will return a list or tuple's length

```
# A list of floats.
grades = [80.0, 75.0, 88.1, 91.0, 50.0]

# Print the length: 5
print(len(grades))

# Print the first element.
print(grades[0])

# Print the last element (badly?).
print(grades[len(grades) - 1]) # Why is there a "- 1"?
```

for Loops and Arrays (General)

- In all programming languages with arrays, *for* loops and arrays are very useful to use together

- Arrays are numerically ordered sets of values
- *for* loops allow you to count through the set of values

```
numbers = [10, 25, 25, 5, 100, 25]
total = 0

# Count through the list and add to the total.
for count in range(0, len(numbers)):
    total += numbers[count]
print(total)
```

DURHAM COLLEGE

for Loops in Python

- What this means is that writing a for loop to iterate through a tuple in Python is super easy!
- When you are working with a tuple, you generally don’t need to use *range*()

```
# a list of professor names
names = ["Kyle", "Thom", "Kamaljeet", "Jen", "Samson"]

# print all professor names in order
for current_name in names:
    print(current_name)
```

DURHAM COLLEGE

Functions That Are Handy with Lists and Tuples

Function	Description
`len()`	Returns the number of elements in the list or tuple
`list()`	Converts something – like a tuple – to a list
`max()`	Returns the highest value item in the list or tuple
`min()`	Returns the lowest value item in the list or tuple

Methods That Are Handy with Lists (but not Tuples)

Function	Description
`append()`	Adds an element to the end of a list
`clear()`	Removes all elements from a list
`index()`	Returns the index of a specified value
`pop()`	Removes the element at a specified index
`remove()`	Removes an element with a specific value
`sort()`	Attempts to sort the contents of the list

The Problem, Starring You!

You have now progressed beyond the midpoint of this course and would like to write a program to calculate your midterm mark. The description on how your midterm mark was calculated is in the course outline. Please refer back to the course outline for details.

Your program must ask the user for their marks for each assessment, ordered by type. The assessments include the Pre-Class Activities, In Class Exercises, and Assignments. Once the user finishes their entry, calculate and display the user's grade as a percentage and their percentage for each type of assessment.

Your user may not always enter the correct details when asked. Ensure your program can identify that and guide the user on entering valid data.

DURHAM COLLEGE

Plan to Program!

- With consideration to the Course Outline, and given that we need the program to work regardless of what the user does, write a plan for a program to determine a midterm grade for this course.

1. Analyze the problem; imagine and discuss validation rules for each of these variables.
2. Make a discussion post including your notes on DC Connect (**COSC 1100** -> **Activities** -> **Discussion** -> **Class Exercises**), including all group members' names.
3. Be prepared to discuss your notes in class.

Class Exercise Part 1 Discussion

- How would you store the user’s entries?
- In what ways are these planning notes helpful compared to trying to write a plan or code directly from the problem description?
- If you’ve reviewed a colleague’s plan, what constructive ways could their notes be improved?

COSC 1100
Class Exercise

Arrays: Learning to Use Arrays Effectively

Part 2 of 2

Arrays:
Learning to Use Arrays Effectively

Goal of Part 2:

Experience writing a program with arrays

DURHAM COLLEGE

Requirements

- Did you come prepared?
  - Did you watch the Lecture videos on arrays?
  - Hopefully you are comfortable with loops!

- Collaboration.
  - Although you did your planning collaboratively and you have access to the posted plans, *everyone will do their own coding today*!
  - Each student must submit their finished application to earn full marks on this week's Class Exercises

The Problem, Starring You! (Review)

You have now progressed beyond the midpoint of this course and would like to write a program to calculate your midterm mark. The description on how your midterm mark was calculated is in the course outline. Please refer back to the course outline for details.

Your program must ask the user for their marks for each assessment, ordered by type. The assessments include the Pre-Class activity, In Class Exercises, and Assignments. Once the user finishes their entry, calculate and display the user's grade as a percentage and their percentage for each type of assessment.

Your user may not always enter the correct details when asked. Ensure your program can identify that and guide the user on entering valid data.

Plan to Program!

1. With consideration to the plan you developed with your group, code the application using Python!
2. Chat, ask lots of questions, and don’t be afraid of mistakes.
3. Have it checked during class, and submit your finished Python code file to DC Connect
   (**COSC 1100** -> **Activities** -> **Assignments** -> **Class Exercise: Arrays**)

This is meant to be done during class time; submissions received after our class time ends may be considered late.

Class Exercise Part 2 Discussion

- Was it harder or easier to do the coding part on your own?
- Do you think you learned more or less from this experience?

- If you coded this with a certain kind of care, you could use the same program to calculate a **final** grade instead of a **midterm** grade by changing just three values in the entire program.
  - Which values?
  - Is this beneficial? Or could this be a bad thing? Why?

Class Exercise Part 2 Summary

- Arrays makes programming and handling larger amounts of data easier

- Is there a part of today's code that you think we might use again?